

Trabajo práctico 4.1

Listas

1. Implemente la clase `NodeSingle` para ser usada en una lista enlazada.
2. Implemente la clase `NodeDouble`, hija de la anterior, para ser usada en una lista doblemente enlazada.
3. Implemente una clase `SingleLinkedList` que, usando la clase definida en el ejercicio 1, incluya las siguientes operaciones.
 - a. `insert(x, p)`
Inserta el elemento `x` en la posición `p` de la lista.
 - b. `delete(x)`
Elimina la primera ocurrencia del elemento `x` de la lista.
4. Suponga que posee una clase `DoublyLinkedList` que está implementada utilizando nodos como los del ejercicio 2. Añada a la clase un método que le permita eliminar el menor elemento de la lista.
5. Implemente una clase `CircularList` con las operaciones que considere más importantes.
6. Implemente una clase `Playlist`, derivada de `CircularList`, que incluya las siguientes operaciones.
 - a. `nuevaCancion(c)`
Añade una canción al final de la lista
 - b. `reset()`
Se prepara para reproducir la primera canción de la lista.
 - c. `avanzar()`
Se prepara para reproducir la siguiente canción de la lista. Si no hay más canciones, sigue por la primera.
 - d. `reproducir()`
Imprime "Reproduciendo " y el nombre de la canción actual de la lista.

Ejemplo:

```
>> lista = Playlist()
>> lista.nuevaCancion("Khazad-dum")
>> lista.nuevaCancion("Valinor")
>> lista.nuevaCancion("Nampat")
>> lista.nuevaCancion("Elendil and Isildur")
>> lista.reset()
>> lista.reproducir()
Reproduciendo Kahazad-dum
>> lista.reproducir()
```

Reproduciendo Valinor

```
>> lista.reproducir
```

Reproduciendo Valinor

```
>> lista.avanzar()
```

```
>> lista.avanzar()
```

```
>> lista.avanzar()
```

```
>> lista.reproducir()
```

Reproduciendo Kahazad-dum

7. Implemente una clase NodeEx derivada de NodeSingle que incorpore las operaciones

- a. unHermanito(self, x)
 - # Al invocar N.unHermanito(y) agrega un nuevo nodo
 - # (con valor y) a la derecha de N
 - # sin perder el resto de los elementos de la lista.
- b. borrarSiguierte(self)
 - #Elimina el elemento siguiente al nodo,
 - # preservando los elementos que siguen.
- c. misAmigos(self, x)
 - # Devuelve True si el elemento x se encuentra en el nodo
 - # o en alguno de los nodos siguientes.
 - # False en caso contrario.

8. Implemente una clase NodeDoubleEx derivada de NodeDouble que incorpore las operaciones

- a. borrarse(self)
 - # Hace que el nodo se elimine a sí mismo de la lista y quede
 - # excluido de ella,
 - # preservando a los demás
- b. rotarADerecha(self)
 - # Hace que el nodo se intercambie con el nodo que está
 - # a su derecha (nodo siguiente).
 - # No debe intercambiar simplemente el valor
 - # sino el objeto nodo completo
- c. multiplicarse(self)
 - # Al invocar N1.multiplicarse() se generan dos copias Ni y Nd
 - # del nodo. Donde:
 - # - Ni queda a la izquierda de N1
 - # - Nd queda a la derecha de N1
 - # - Ni tiene el mismo valor que N1 pero decrementado en 1
 - # - Nd tiene el mismo valor que N1 pero aumentado en 1